

Prompt Ladder

Backend API Prompt Engineering

A controlled six-run experiment showing how one prompt changes when a single engineering layer is added at each step.

Track	Backend AI Engineering
Task	Prompt Ladder
Runs	6 total: baseline + 5 versions
Method	One change per version, output compared after every run

Experiment objective

The goal was not to make the prompt longer for its own sake. Each version was designed to fix the most visible weakness in the previous output while keeping every other part unchanged. This makes it possible to see which prompt ingredient actually earned its place.

Ladder Overview

Run	Layer added	Main observed effect
Baseline	None	Model could not act and asked for clarification.
V1	Clearer goal	Model immediately built a complete task API, but guessed the stack and design.
V2	Real context	Added product-level observations, but barely changed the implementation.
V3	Specified output format	Made the response much easier to review and use.
V4	Constraints	Changed the implementation to the intended Python/FastAPI stack.
V5	Verification requirements	Added concrete endpoint-by-endpoint test evidence.

Comparison rule

Every version added exactly one named layer. Outputs were compared against the immediately previous run rather than against memory or against the final result.

Baseline - Weak Prompt

Layer added: None

Prompt

Write backend code.

Representative output excerpt

"I'd love to help - but backend code could mean a lot of things. Let me narrow it down quickly. What should this backend do?"

Run notes

What changed in the prompt	Nothing. This was the intentionally weak baseline prompt.
What actually improved in the output	Nothing useful was produced yet. The model could not start the task and instead asked for clarification.
What still failed	The prompt did not define the goal, domain, framework, functionality, or expected result.
What I would try next	Add one clear goal so the model knows what backend to build.

Version 1 - Clearer Goal

Layer added: Clearer goal

Prompt

Build a backend API for managing tasks.

Representative output excerpt

The model built a working Node.js/Express task API with CRUD operations, filtering, validation, centralized error handling, a README, and run instructions. It explicitly stated that it had made default choices such as Node/Express, in-memory storage, and task status values.

Run notes

What changed in the prompt	The vague request was replaced with a specific goal: build a backend API for managing tasks.
What actually improved in the output	The model stopped asking what kind of backend was needed and produced a complete, runnable implementation with CRUD endpoints, validation, filtering, error handling, and setup instructions.
What still failed	The model had to guess major implementation choices such as Node.js, Express, in-memory storage, and the status model.
What I would try next	Add real context about the application so the model has more information before choosing architecture and behavior.

Version 2 - Real Context

Layer added: Real context

Prompt

Build a backend API for managing tasks.

The API is for a small productivity application where users need to create, read, update, and delete tasks.

Representative output excerpt

The model mostly reused the previous implementation and explained that it already matched the productivity-app use case. It added two useful product observations: tasks were global rather than user-specific, and the in-memory data would disappear after restart.

Honest observation: This layer did not help much with the implementation itself.

Run notes

What changed in the prompt	Context was added explaining that the API belongs to a small productivity application where users create, read, update, and delete tasks.
What actually improved in the output	The improvement was small. The response connected the API to the product use case and surfaced two product-level concerns: task ownership and persistence.
What still failed	The implementation did not materially improve. The model kept the same architecture and mostly repeated the previous solution.
What I would try next	Specify the output format so the response is organized around exactly what needs to be reviewed and used.

Version 3 - Specified Output Format

Layer added: Specified output format

Prompt

Build a backend API for managing tasks.

The API is for a small productivity application where users need to create, read, update, and delete tasks.

Return:

1. The complete backend code
2. A short explanation of each endpoint
3. The commands needed to install dependencies and run the server

Representative output excerpt

The response was divided into three clear sections: complete backend code, an endpoint table, and install/run commands. The implementation itself remained Node.js/Express, but the deliverable became much easier to inspect and reuse.

Run notes

What changed in the prompt	The response structure was specified: complete code, endpoint explanations, and installation/run commands.
What actually improved in the output	The response became much easier to review and use. Code, API behavior, and setup instructions were separated cleanly instead of being mixed in a general answer.
What still failed	The model still chose Node.js/Express and continued making architecture and data-model decisions because the technology had not been constrained.
What I would try next	Add technical constraints so the implementation uses the exact language, framework, and storage approach required.

Version 4 - Constraints

Layer added: Constraints

Prompt

Build a backend API for managing tasks.

The API is for a small productivity application where users need to create, read, update, and delete tasks.

Return:

1. The complete backend code
2. A short explanation of each endpoint
3. The commands needed to install dependencies and run the server

Constraints:

- Use Python and FastAPI
- Use in-memory storage only
- Do not use a database
- Keep the implementation simple and readable

Representative output excerpt

The output switched to Python/FastAPI, used Pydantic models, produced a minimal requirements file, kept storage in memory, and exposed Swagger automatically at /docs. Invalid input used FastAPI's default 422 behavior.

Run notes

What changed in the prompt	Technical constraints were added for Python, FastAPI, in-memory storage, no database, and a simple readable implementation.
What actually improved in the output	The implementation now matched the intended technology stack instead of relying on the model's assumptions. It switched from Node.js/Express to Python/FastAPI and preserved the requested storage model.
What still failed	The model still made behavior decisions that were not explicitly verified. For example, invalid request data returned FastAPI's default 422 response.
What I would try next	Add verification requirements so the model checks the API against specific functional and HTTP behavior before finalizing.

Version 5 - Verification Requirements

Layer added: Verification requirements

Prompt

Build a backend API for managing tasks.

The API is for a small productivity application where users need to create, read, update, and delete tasks.

Return:

1. The complete backend code
2. A short explanation of each endpoint
3. The commands needed to install dependencies and run the server

Constraints:

- Use Python and FastAPI
- Use in-memory storage only
- Do not use a database
- Keep the implementation simple and readable

Before finalizing, verify that:

- GET /tasks works
- GET /tasks/{id} works
- POST /tasks works
- PUT /tasks/{id} works
- DELETE /tasks/{id} works
- Missing task IDs return 404
- Invalid input returns an appropriate client error
- The API can be tested through Swagger UI

Representative output excerpt

The response added a concrete verification table: GET /tasks 200, GET /tasks/{id} 200, POST 201, PUT 200, DELETE 204, missing IDs 404, invalid input 422, /docs 200, and /openapi.json 200. The response now showed evidence rather than only saying the code had been tested.

Run notes

What changed in the prompt	An explicit verification checklist was added for every CRUD endpoint, missing-resource behavior, invalid input, and Swagger UI.
What actually improved in the output	The response included concrete verification results instead of a general claim. Actual status codes were reported for each endpoint, missing IDs, invalid input, Swagger UI, and the OpenAPI schema.
What still failed	The model treated FastAPI's default 422 validation response as acceptable because the requirement only said "appropriate client error." Exact error behavior would still need to be specified if 400 were required.
What I would try next	For production work, make the verification criteria more precise by specifying exact expected status codes, response schemas, and edge cases.

Final Reusable Prompt

This version keeps the layers that proved useful while making the request self-contained enough for another backend developer or AI engineer to reuse without extra context.

Build a backend API for managing tasks for a small productivity application.

The API must let clients create, read, update, and delete tasks.

Return:

1. The complete backend code
2. A short explanation of each endpoint
3. The commands needed to install dependencies and run the server
4. A short verification report showing the result of each required check

Constraints:

- Use Python 3.10+ and FastAPI
- Use in-memory storage only
- Do not use a database or external persistence
- Keep the implementation simple, readable, and easy to run locally
- Use clear request/response models and sensible HTTP status codes
- Expose interactive Swagger documentation at /docs

Required behavior:

- GET /tasks lists tasks
- GET /tasks/{id} returns one task
- POST /tasks creates a task
- PUT /tasks/{id} updates a task
- DELETE /tasks/{id} deletes a task
- Missing task IDs return 404
- Invalid request bodies return a clear client-error response

Before finalizing, run or otherwise verify each endpoint and report the observed HTTP status code. Also verify that /docs loads successfully and that the OpenAPI schema is available.

Why these layers stayed

Clearer goal	Turned an unusable request into an actionable backend task.
Real context	Added some product awareness, although its implementation impact was limited.
Output format	Made the result easier to inspect, run, and hand to someone else.
Constraints	Prevented the model from choosing the wrong stack and storage approach.
Verification requirements	Changed the answer from a claim of correctness into observable evidence.

Conclusion

The largest gains did not come from simply adding more words. The clearer goal made the task actionable, constraints corrected the technology choice, and verification requirements produced the strongest increase in trustworthiness. The real-context step was useful for product thinking but had little effect on the implementation, which is an important reminder that not every additional prompt layer earns its place equally.